



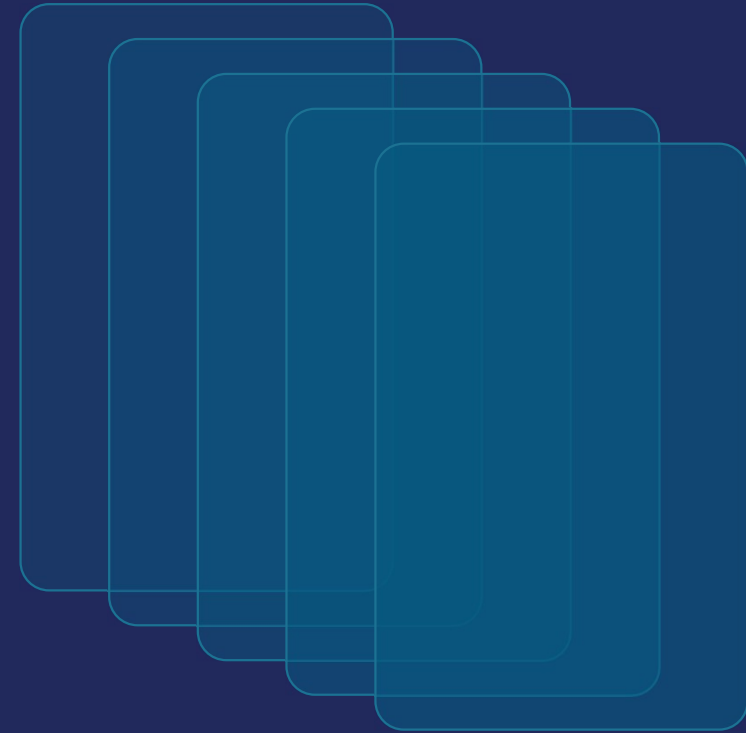
KUBERNETES TRAINING

Module 3:

Pods, Pod Lifecycle & Services

Build the mental model from one Pod to stable Service-based communication.

2026 MODERNIZED EDITION



From Docker + kubectl to Pods



Module 1

Images package software.
Containers run those images.
Docker gives the low-level runtime workflow.



Module 2

Control plane decides.
Worker nodes execute.
kubectl talks to the API server.

TODAY

1 Pod = smallest deployable unit

2 Pod lifecycle + failure states

3 Create / inspect / debug Pods

4 First YAML manifests

5 Service = stable network identity

6 Service discovery + access



What is a Pod?

The smallest and simplest unit you create to run containers.



shared network namespace
shared localhost
shared volumes when configured

- One or more containers
- Shared network + storage resources
- Ports, env, volumes, security config



Why Pods, not just containers?



Kubernetes view

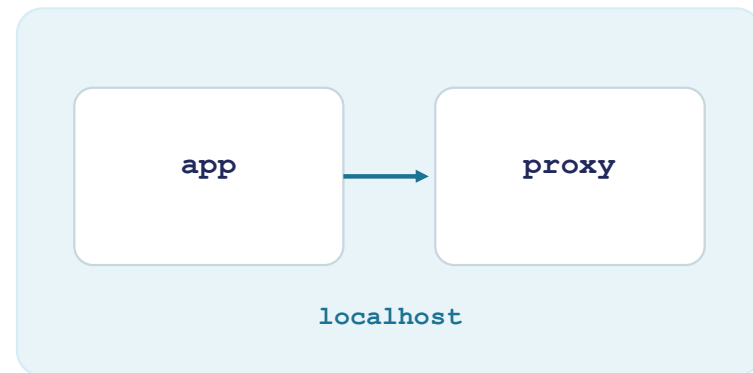
Same-Pod containers can share volumes

Same-Pod containers share localhost

Pod = scheduling + networking boundary

Container = process packaging

ONE POD, ONE NETWORK SPACE



Pod-to-Pod communication is possible across the cluster network by default; service names become the stable discovery mechanism introduced later.



Pod-to-Pod communication

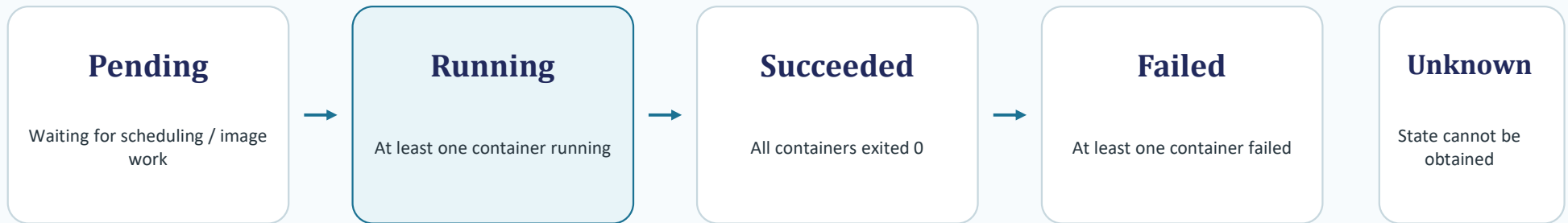
Each Pod receives a cluster-internal IP. Other Pods can reach it directly inside the cluster.



Pod IPs are useful for direct cluster traffic — but they are ephemeral. That is exactly why Services exist.



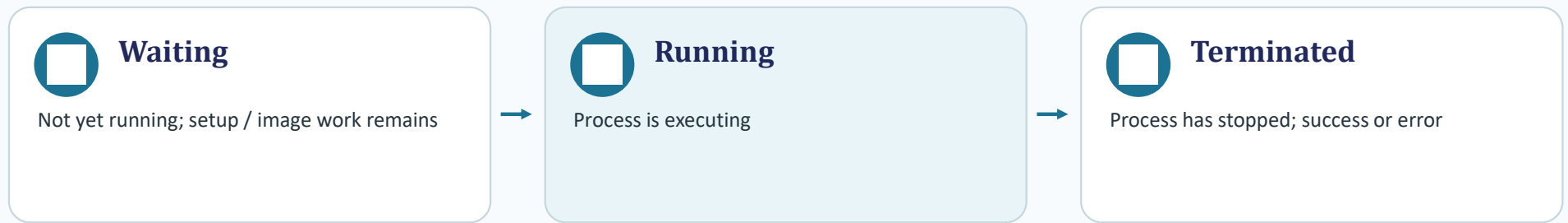
Pod lifecycle — phases



Lifecycle is the Pod phase. Container-level states are a separate layer.



Container-level states roll up to the Pod



Pod phase is a summary of what is happening across its containers.

Running → at least one container is running. | Succeeded → all terminated successfully. | Failed → at least one terminated in failure.



Debugging Pod failures & restart issues

🔴 🟡 🟢 \$ Terminal

```
kubectl describe pod nginx-pod
kubectl logs nginx-pod
```

🔴 🟡 🟢 \$ Terminal

```
kubectl get events
kubectl get pods --watch
```

CRASH → BACKOFF → DEBUG

- 1 Container crashes
- 2 Restart policy applies
- 3 Backoff delay increases
- 4 CrashLoopBackOff may appear
- 5 Inspect logs + Events + describe

CrashLoopBackOff is a restart-delay signal — not the root cause. Find the cause in logs, Events, and describe output.



Create a Pod imperatively — `kubectl run`

A fast way to create a single Pod for learning, diagnostics, or quick experiments.

Terminal

```
kubectl run nginx-pod --image=nginx:1.28-alpine
```

Terminal

```
kubectl get pods
```

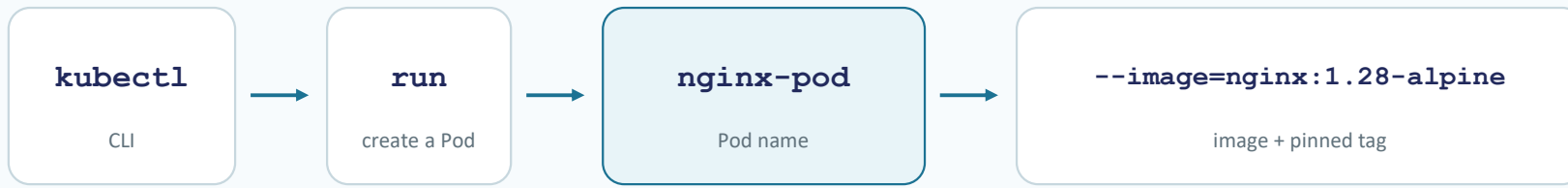
OUTPUT

```
pod/nginx-pod created
```

OUTPUT

NAME	READY	STATUS	RESTARTS	AGE
nginx-pod	1/1	Running	0	8s

Anatomy of `kubectl run`



 \$ Terminal

```
kubectl run <pod-name> --image=<container-image>:<tag>
```

Pin the image tag so the example is reproducible.



Inspecting Pods — get + wide

Terminal

```
kubectl get pods
```

OUTPUT

NAME	READY	STATUS	RESTARTS	AGE
nginx-pod	1/1	Running	0	3m

Terminal

```
kubectl get pod nginx-pod -o  
wide
```

OUTPUT

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
nginx-pod	1/1	Running	0	3m	10.244.2.2	lab-worker2

Use `-o wide` when you need the Pod IP and node placement — the clues that matter for network/debug work.



Deep inspection — kubectl describe pod

Terminal

```
kubectl describe pod nginx-  
pod
```

OUTPUT

```
Name:      nginx-pod  
Namespace: default  
Node:      lab-worker2/172.19.0.3  
Status:    Running  
IP:        10.244.2.2  
Containers:  
  nginx-pod: nginx:1.28-alpine Running  
Events:  
  Normal   Scheduled   Successfully assigned  
  Normal   Pulled      Container image pulled  
  Normal   Started     Container started
```

Namespace default

IP 10.244.2.2

Events scheduled / pulled /
started



Communicating with a Pod internally

A Pod IP is private to the cluster. Test it from another Pod, not from your laptop.

Terminal

```
kubectl get pod nginx-pod -o wide
```

OUTPUT

NAME	IP	NODE
nginx-pod	10.244.2.2	lab-worker2

Terminal

```
kubectl run client \
  --image=busybox:1.37 -- sleep 3600
kubectl exec client -- \
  wget -qO- http://10.244.2.2
```

OUTPUT

```
pod/client created

<!DOCTYPE html>
<html>
<head><title>Welcome to nginx!</title></head>
```



Viewing container output — logs

Terminal

```
kubectl logs nginx-pod
```

Terminal

```
kubectl logs -f nginx-pod
```

OUTPUT

```
10.244.1.20 - - [02/Sep/2026:12:01:11 +0000] "GET / HTTP/1.1" 200
10.244.1.20 - - [02/Sep/2026:12:01:12 +0000] "GET /favicon.ico
HTTP/1.1" 404
```

OUTPUT

```
10.244.1.20 - - [02/Sep/2026:12:02:04 +0000] "GET / HTTP/1.1" 200
10.244.1.20 - - [02/Sep/2026:12:02:09 +0000] "GET /health HTTP/1.1" 200
... streaming
```

-f = follow. Keep the terminal open and watch new log lines arrive.



Deleting Pods

🔴 🟡 🟢 \$ Terminal

```
kubectl delete pod nginx-pod
```

OUTPUT

```
pod "nginx-pod" deleted from default namespace
```

Force deletion (use sparingly)

```
kubectl delete pod nginx-pod --force --grace-period=0
```

Normal deletion is the default. Force deletion is a troubleshooting escape hatch, not the everyday workflow.



Your First YAML Manifest



A Pod manifest describes the desired object — not a sequence of shell steps.

Milestone: this is the course's first declarative Kubernetes YAML.

apiVersion → kind → metadata → spec

Save it as pod.yaml, then apply it with kubectl.

Pod manifest — pod.yaml

```
pod.yaml

apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: nginx
spec:
  containers:
    - name: nginx
      image: nginx:1.28-alpine
      ports:
        - containerPort: 80
```

LABEL = app: nginx

The Service later selects this exact label.

GOOD HABIT

Pin image tags instead of relying on :latest.



Apply the Pod YAML

Terminal

```
kubectl apply -f pod.yaml
```

OUTPUT

```
pod/nginx-pod created
```

Terminal

```
kubectl get pods
```

OUTPUT

NAME	READY	STATUS	RESTARTS	AGE
nginx-pod	1/1	Running	0	7s

Imperative: “run this.” Declarative: “make the cluster match this file.”



Why Services?

Pod IPs are ephemeral. A Pod can die, be recreated, and come back with a different IP.

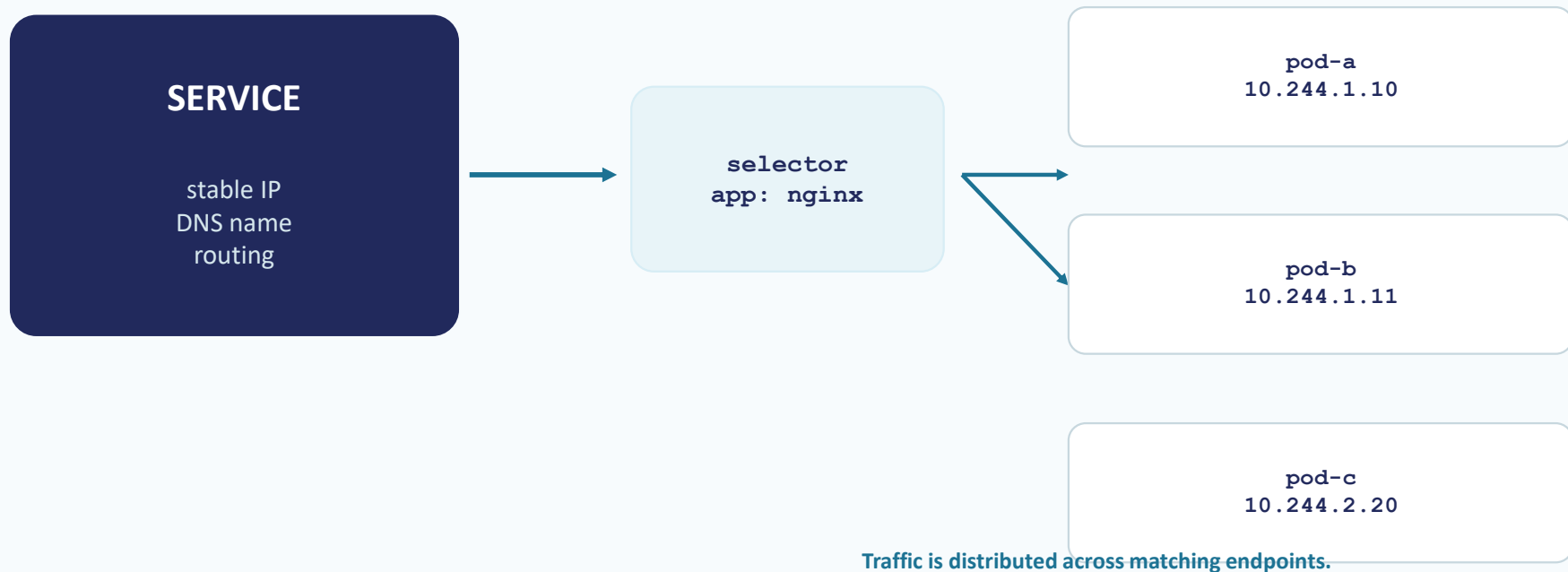


Service = stable network identity

Stable IP + DNS name + routing to matching Pods



What is a Service?





Create a Service imperatively — `kubectl expose`

Terminal

```
kubectl expose pod nginx-pod \  
  --type=NodePort --port=80 \  
  --name=nginx-nodeport
```

OUTPUT

```
service/nginx-nodeport exposed
```

Terminal

```
kubectl get services
```

OUTPUT

NAME	TYPE	CLUSTER-IP	PORT(S)
nginx-nodeport	NodePort	10.96.12.57	80:32353/TCP

Service manifest — service.yaml

```
service.yaml

apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
spec:
  type: ClusterIP
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
```

SELECTOR MATCH

app: nginx

PORT MAPPING

Service 80 → Pod 80



Service types — choose the exposure you need

ClusterIP

Inside cluster

Internal only

Default service type. Stable virtual IP for in-cluster traffic.

NodePort

Node network

Node IP + high port

Exposes the Service on a port across cluster nodes.

LoadBalancer

Outside cluster

External endpoint

Cloud / provider integration for external access.

Start with ClusterIP for service-to-service traffic; expose externally only when the use case requires it.



Accessing a Pod via a Service

Terminal

```
kubectl get services
```

OUTPUT

NAME	TYPE	CLUSTER-IP	PORT(S)
nginx-svc	ClusterIP	10.96.189.200	80/TCP

Terminal

```
kubectl describe service nginx-svc
```

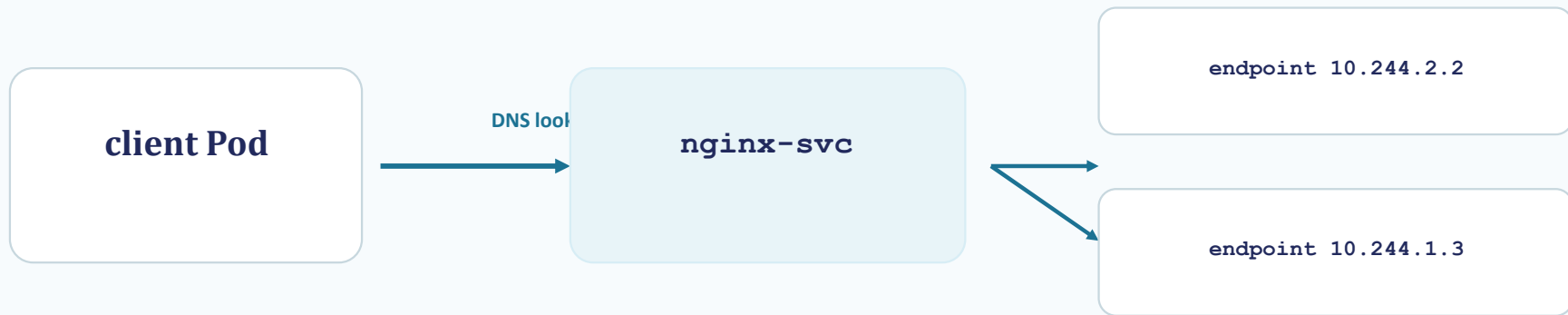
OUTPUT

```
Name:      nginx-svc
Type:      ClusterIP
IP:        10.96.189.200
Port:      80/TCP
TargetPort: 80/TCP
Endpoints: 10.244.2.2:80
```



Service discovery — use names, not Pod IPs

Kubernetes DNS lets clients address a Service by name while the Service tracks changing Pod endpoints.



Terminal

```
kubectl exec client -- wget -qO- http://nginx-svc
```

The Service name stays stable even when Pod endpoints change.



Cleaning up — remove the Service and Pods

Terminal

```
kubectl delete service nginx-svc
```

OUTPUT

```
service "nginx-svc" deleted from  
default namespace
```

Terminal

```
kubectl delete pod nginx-pod  
kubectl delete pod client
```

OUTPUT

```
pod "nginx-pod" deleted from default namespace  
pod "client" deleted from default namespace
```

Terminal

```
kubectl get pods
```

OUTPUT

```
No resources found in default namespace.
```



Try It Yourself — Pod + Service lab

1

Create

```
kubectl apply -f pod.yaml
```

```
pod/nginx-pod created
```

2

Expose

```
kubectl apply -f service.yaml
```

```
service/nginx-svc created
```

3

Test

```
kubectl exec client -- wget -qO- nginx-svc
```

```
<title>Welcome to nginx!</title>
```

4

Clean

```
kubectl delete service nginx-svc
```

```
service deleted
```

Use the kind cluster from Module 2. The selector in service.yaml must match app: nginx in pod.yaml.



Generate YAML instead of typing it

Let kubectl write the first draft: `--dry-run=client -o yaml` prints a manifest without creating anything.

Terminal

```
k run nginx-pod --image=nginx:1.28-alpine \
-l app=nginx --dry-run=client -o yaml
```

OUTPUT

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    app: nginx
...
```

Terminal

```
k run nginx-pod ... -o yaml > pod.yaml
kubectl apply -f pod.yaml
```

OUTPUT

```
pod/nginx-pod created

# Same result as typing the YAML by hand -
# but no typos and no indentation mistakes.
```



Watch the lifecycle happen

A Pod that does one job and exits walks through every phase: Pending → Running → Succeeded.

Terminal

```
k run short-job --image=busybox:1.37 \
  --restart=Never -- sh -c 'sleep 5'
k get pod short-job -w
```

Terminal

```
k get pod short-job \
  -o jsonpath='{.status.phase}'
```

OUTPUT

NAME	READY	STATUS	AGE
short-job	0/1	ContainerCreating	1s
short-job	1/1	Running	1s
short-job	0/1	Completed	7s

OUTPUT

Succeeded

```
# exit code 0 + restartPolicy Never
# → phase Succeeded (exit 3 → Failed)
```



Break it on purpose — and read the clues

Terminal

```
kubectl get pods
kubectl logs crasher
kubectl describe pod crasher
```

OUTPUT

```
NAME      READY   STATUS             RESTARTS
bad-image  0/1     ImagePullBackOff    0
crasher   0/1     CrashLoopBackOff    3

$ kubectl logs crasher
starting app...
ERROR: DATABASE_URL is not set

$ kubectl describe pod crasher
  Last State:      Terminated
    Exit Code:      1
  Restart Count:   3
Warning BackOff    Back-off restarting failed container
```

Pull fails wrong tag / name

Crash loop exit code 1

Root cause read the logs



Service discovery with DNS

Every Service gets a DNS name. Pods look it up through CoreDNS — Pods themselves get no name.

Terminal

```
kubectl exec client -- \
  wget -qO- http://nginx-svc
```

OUTPUT

```
<title>Welcome to nginx!</title>

# short name works inside the same namespace
```

Terminal

```
kubectl exec client -- nslookup \
  nginx-svc.default.svc.cluster.local
kubectl exec client -- \
  wget -qO- http://nginx-pod
```

OUTPUT

```
Server:      10.96.0.10
Name:        nginx-svc.default.svc.cluster.local
Address:     10.96.189.200

wget: bad address 'nginx-pod'
```



Labels decide which Pods a Service uses

Terminal

```
kubectl label pod nginx-pod app=web --overwrite
```

OUTPUT

NAME	PORTS	ENDPOINTS
nginx-svc-n2v26	<unset>	<unset>

Terminal

```
kubectl exec client -- \
  wget -qO- -T 3 http://nginx-svc
```

OUTPUT

```
wget: can't connect to remote host
(10.96.189.200): Connection refused
# the Service has no endpoints now
```

Terminal

```
k label pod nginx-pod app=nginx --overwrite
```

OUTPUT

```
nginx-svc-n2v26    80    10.244.2.2    ← back!
```



The Service survives Pod replacement

Terminal

```
kubectl delete pod nginx-pod
kubectl apply -f pod.yaml
k get pod nginx-pod -o wide
```

OUTPUT

```
old IP: 10.244.1.2
new IP: 10.244.1.3
# a new Pod always gets a new IP
```

Terminal

```
k get endpointslices \
-l kubernetes.io/service-name=nginx-svc
k exec client -- wget -qO- nginx-svc
```

OUTPUT

NAME	PORTS	ENDPOINTS
nginx-svc-n2v26	80	10.244.1.3


```
<title>Welcome to nginx!</title>
```



Two containers, one Pod

two-containers.yaml (trimmed)

```
spec:
  volumes:
    - name: shared-html
      emptyDir: {}
  containers:
    - name: web
      image: nginx:1.28-alpine
      volumeMounts:
        - name: shared-html
          mountPath: /usr/share/nginx/html
    - name: writer
      image: busybox:1.37
      # writes index.html every 5 s
      volumeMounts:
        - name: shared-html
          mountPath: /html
```

SHARED localhost

writer calls `http://localhost:80` — nginx logs it from 127.0.0.1.

SHARED VOLUME

web serves the file that writer just wrote.



Try the 2026 features: sidecar + resize

Terminal

```
k apply -f sidecar-resize.yaml
k get pod modern-pod
```

OUTPUT

NAME	READY	STATUS	RESTARTS
modern-pod	2/2	Running	0

2/2 = web + native sidecar (log-shipper)

Terminal

```
k patch pod modern-pod \
  --subresource resize --patch ...
```

OUTPUT

```
pod/modern-pod patched
before: cpu limit=200m  restarts=0
after:  cpu limit=400m  restarts=0
# resized in place — no restart
```

Native sidecars (stable v1.33) and in-place resize (stable v1.35) — both verified here on Kubernetes v1.37.



Pods & Services in 2026 — what's new



Native sidecar containers

Stable since Kubernetes v1.33. A sidecar is modeled as a restartable init container using `restartPolicy: Always` — useful for logging, proxies, monitoring, and similar helpers. Full sidecar patterns come later in the course.

```
pod.yaml
initContainers:
- name: log-proxy
  restartPolicy: Always
```



In-Place Pod Resize

Stable since Kubernetes v1.35. CPU and memory requests / limits for containers in a running Pod can be changed without recreating the Pod; restart behavior is controlled by `resizePolicy`.

```
concept
CPU / memory → mutable
Pod stays in place
```

Modernization note: these features extend the same Pod abstraction you are learning here — they do not change the core Pod + Service mental model.



Lab: user namespaces — root that isn't root

Terminal

```
kubectl apply -f userns-pod.yaml # hostUsers: false
kubectl run normal-pod --image=busybox:1.37 \
  -- sleep 3701
```

OUTPUT

```
pod/userns-pod created
pod/normal-pod created # no hostUsers line
```

Terminal

```
kubectl exec userns-pod -- id
kubectl exec userns-pod -- cat /proc/self/uid_map
```

OUTPUT

```
uid=0(root) ...      ← root inside
0 4153409536 65536    ← mapped range
```

Terminal

```
# on the node: which UID runs each sleep?
docker exec lab-worker stat -c %u /proc/<pid>
```

OUTPUT

```
userns-pod → host UID 4153409536
normal-pod → host UID 0    (real root!)
```

Common pitfalls — Pod + Service troubleshooting



Selector mismatch

Service selector: app: nginx
Pod label: app: web

No matching endpoints.



Wrong port

Service port: 80
targetPort: 8080

Traffic reaches the Service but not the app port.



Pod IP from outside

`curl 10.244.x.x from laptop`

Pod IP is cluster-internal. Test from another Pod.



CrashLoopBackOff

`kubectl logs + describe + events`

Backoff is a symptom; inspect the application failure.



Module recap — from Pod to Service



Pod

Smallest deployable unit; one or more containers.



Lifecycle

Pending → Running → Succeeded / Failed / Unknown.



Debug

get → describe → logs → events → watch.



YAML

apiVersion / kind / metadata / spec.



Service

Stable IP + DNS + routing to matching Pods.



Discovery

Use Service names; Pod IPs are ephemeral.

Next step: manage these Pods with controllers — instead of managing individual Pods by hand.



KNOWLEDGE CHECK

9 questions

Before you move on

Real Pod and Service problems, with the terminal output you'd see.
Answer key on the last quiz slide.

POD

runs

SERVICE

routes

NEXT

manages



Question 1 of 9

Two containers in the same Pod need to talk to each other. Which address should the app use?

Scenario

```
k get pod two-containers  
READY 2/2 Running
```

Think about

One Pod, one network

OUTPUT

```
containers: web (port 80)  
            writer
```

CHOOSE ONE

- A. The other container's Pod IP from `kubectl get pods`
- B. `localhost` and the other container's port
- C. The Service name of the Pod
- D. They can't — containers in a Pod are isolated



Question 2 of 9

An app hard-codes another Pod's IP. After that Pod is recreated, the app breaks. What's the right fix?

🔴 🟡 🟢 \$ Scenario

```
k get pod nginx-pod -o wide
```

OUTPUT

```
before: 10.244.1.2  
after:  10.244.1.3
```

🔴 🟡 🟢 Think about

Pod IPs are ephemeral

CHOOSE ONE

- A. Put a Service in front and use its DNS name
- B. Give the Pod a static IP with a special annotation
- C. Never delete Pods
- D. Restart the app every time the IP changes



Question 3 of 9

The Service exists, but it has no endpoints and clients get “connection refused”. Most likely cause?

🔴 🟡 🟢 \$ Scenario

```
k get endpointslices \
-l kubernetes.io/service-name=web
```

OUTPUT

NAME	PORTS	ENDPOINTS
web-x7k2p	<unset>	<unset>

🔴 🟡 🟢 Think about

Labels + selectors

CHOOSE ONE

- A. The cluster's DNS is down
- B. The Pod IP changed
- C. The Service selector doesn't match the Pod's labels
- D. The Service type must be NodePort



Question 4 of 9

A Pod shows CrashLoopBackOff and the restart count keeps rising. What should you do first?

🔴 🟡 🟢 \$ Scenario

```
k get pods
```

OUTPUT

NAME	READY	STATUS	RESTARTS
api	0/1	CrashLoopBackOff	5

🔴 🟡 🟢 Think about

Backoff is a symptom

CHOOSE ONE

- A. Delete the Pod and create it again
- B. Force-delete it with `--grace-period=0`
- C. Wait — it will fix itself
- D. Read `kubectl` logs and describe to find why the container exits



Question 5 of 9

A new Pod is stuck in ImagePullBackOff. What is the most likely fix?

🔴 🟡 🟢 \$ Scenario

```
k describe pod web
```

OUTPUT

```
Failed to pull image "nginx:9.99":  
not found  
Error: ImagePullBackOff
```

🔴 🟡 🟢 Think about

Read the event message

CHOOSE ONE

- A. Give the node more memory
- B. Fix the image name or tag (or the registry credentials)
- C. Change the Service type
- D. Add more replicas



Question 6 of 9

curl to a Pod IP works from another Pod, but times out from your laptop. Why?

🔴 🟡 🟢 \$ Scenario

```
# from your laptop
curl 10.244.2.2
```

OUTPUT

```
curl: (28) Connection timed out
# from the client Pod: 200 OK
```

🔴 🟡 🟢 Think about

Cluster network vs your laptop

CHOOSE ONE

- A. The Pod is broken
- B. nginx blocks laptops
- C. Pod IPs are only reachable inside the cluster network
- D. You need to add -o wide



Question 7 of 9

A Service has port: 80 and targetPort: 8080, but the app listens on port 80. What happens?

🔴 🟡 🟢 \$ Scenario

```
k describe service web
```

OUTPUT

```
Port:          80/TCP
TargetPort:    8080/TCP
Endpoints:     10.244.1.5:8080
```

🔴 🟡 🟢 Think about

port vs targetPort

CHOOSE ONE

- A. Traffic reaches the Service but not the app — set targetPort to 80
- B. It works: Kubernetes finds the right port
- C. The Service is rejected by the API server
- D. Only NodePort Services use targetPort



Question 8 of 9

A Pod runs one container with restartPolicy: Never. The container exits with code 0. What is the Pod's phase?

Scenario

```
k get pod short-job
```

OUTPUT

NAME	READY	STATUS	RESTARTS
short-job	0/1	Completed	0

Think about

Exit code 0 = success

CHOOSE ONE

- A. Running
- B. Failed
- C. Unknown
- D. Succeeded



Question 9 of 9

A Pod in namespace shop must call the Service orders in namespace backend. Which name works?

🔴 🟡 🟢 \$ Scenario

```
k exec client -n shop -- \
  wget -qO- http://orders
```

OUTPUT

```
wget: bad address 'orders'
```

🔴 🟡 🟢 Think about

DNS search domains

CHOOSE ONE

- A. orders — short names work everywhere
- B. The Pod IP of an orders Pod
- C. orders.backend (or orders.backend.svc.cluster.local)
- D. backend/orders



Answer key

● ● ● Score yourself

9 questions
1 point each

ANSWERS

Q1	B	same Pod = localhost	→ slide 3 • Step 13
Q2	A	use a Service name	→ slide 19 • Step 11
Q3	C	selector ≠ labels	→ slide 32 • Step 10
Q4	D	logs + describe first	→ slide 8 • Step 7
Q5	B	fix the image name or tag	→ slide 30 • Step 6
Q6	C	Pod IPs are cluster-internal	→ slide 13 • Step 4
Q7	A	targetPort must match the app	→ slide 22 • Step 8
Q8	D	exit 0 + Never = Succeeded	→ slide 6 • Step 5
Q9	C	name.namespace across spaces	→ slide 25 • Step 9

8–9

Module 4 ready

6–7

re-read misses

≤ 5

redo Steps 5–12



MODULE 3 COMPLETE

Next: Module 4

Namespaces, Deployments, ReplicaSets & Rolling Updates

You can now create a Pod, inspect its lifecycle, debug it, and give it a stable network identity.

POD

runs

SERVICE

routes

NEXT

manages